NOTEBOOK GUIDE

The OptiChain notebook, cell by cell

Every one of the 105 cells of *notebooks/OptiChain_VN2_Pipeline.ipynb* explained in simple words: 52 code cells and 53 text cells, with a short look at each cell's code and its real output from the last run.

How to read this guide

Box	What it shows
What it does	the cell's job, in plain words
What you see	the result it prints or draws, as in the last run
Why it matters	why this step is in the project
Code (start)	the first lines of the cell's code, for reference
Output	the real printed output; charts are shown as small pictures
Text cell	a short summary of the explanation written in the notebook

Short vocabulary. *Store-product* = one product in one store (599 in total). *Week 0* = 8 April 2024, the last week of history. *Clean demand* = sales with out-of-stock and pre-launch weeks removed. *NaN* = an empty (missing) value. *p-value* = the chance of seeing the result if the test's starting assumption were true; below 0.05 means "reject it".

Cell 0 · TEXT CELL

The title page of the notebook: the business problem, the data, and a table of all 20 steps.

0 · Setup

Cell 1 · TEXT CELL

Heading: step 0, setup.

Cell 2 · CODE CELL

What it does. Loads the tools (pandas for tables, numpy for maths) and the project's own modules, each with a short comment: D = data, E = evaluation, I = inventory, M = models, P = planning, plots = charts, TS = time series. It reads the settings from config.yaml, creates the artifacts folder, fixes the random seed, hides statsmodels' messages and sets one chart style.
What you see. Nothing is printed.
Why it matters. Fixing the seed (42) means every run gives exactly the same numbers. Hiding the warnings keeps the output readable.

Code (start)

```
import json
import warnings

import numpy as np
import pandas as pd

from optichain import data as D            # loading and cleaning the VN2 files
… (14 more lines)
```

1 · Data collection

Cell 3 · TEXT CELL

Heading: step 1, data collection. A table lists the competition files and what each one contains.

Cell 4 · CODE CELL

What it does. Checks that all VN2 files are in data/raw/vn2. Any missing file is downloaded from the organiser's GitHub page, and reports what it downloaded.
What you see. "Downloaded: nothing, every file is already here".
Why it matters. The notebook works on any computer: if the data is missing, it fetches it itself.

Code (start)

```
fetched = D.download_vn2(cfg.paths.raw_dir, cfg.paths.source_url)   # downloads only the files that are missing
print("Downloaded:", fetched or "nothing, every file is already here")
```

Output

```
Downloaded: nothing, every file is already here
```

Cell 5 · CODE CELL

What it does. Reads all the files into one object called panel: sales, in-stock flags, product groups and the starting stock. It also makes hist, the same data as fast number arrays for the models, and t0, the position of week 0 (8 April 2024).
What you see. 599 store-products × 165 weeks, and the first 5 rows × 8 weeks of sales.
Why it matters. Each row is one product in one store; each column is one week. Everything later works on this table.

Code (start)

```
panel = D.load_panel(cfg.paths.raw_dir, cfg.competition.week_0)    # sales, in-stock flags, groups, starting st
ock
hist = M.History.from_panel(panel)                                  # the same data as arrays, for the models
t0 = panel.t0                                                       # column of week 0 (8 April 2024)
print(f"{panel.sales.shape[0]} store-products x {panel.sales.shape[1]} weeks")
panel.sales.iloc[:5, :8]
```

Output

```
599 store-products x 165 weeks
```

Cell 6 · TEXT CELL

Text: explains the shape of the data (rows are store-products, columns are weeks).

Cell 7 · CODE CELL

What it does. Shows the first rows of three more tables: the in-stock flags (True/False), the product and store groups, and the stock on the shelf and on its way at the start.
What you see. Three small tables.
Why it matters. To see what each file looks like before using it.

Code (start)

```
display(panel.in_stock.iloc[:5, :8])    # True = stock was left at the end of the week
display(panel.master.head())            # product and store groups
panel.initial.head()                    # stock on the shelf and on its way at the start
```

Output: 3 table(s), shown in the notebook.

Cell 8 · TEXT CELL

Heading: step 1.2, the data-quality audit.

Cell 9 · CODE CELL

What it does. Runs a quality check and puts the results in one table: number of stores, products and weeks, missing values, negative values, decimals, share of zero weeks, share of out-of-stock weeks.
What you see. A table of facts: 0 missing, 0 negative, 0 decimals, 43% zero weeks, about 10% out-of-stock weeks.
Why it matters. If the data were broken, every later result would be wrong. Checking first is basic good practice.

Code (start)

```
audit = D.audit(panel)
audit
```

Output: 1 table(s), shown in the notebook.

Cell 10 · CODE CELL

What it does. Opens all 9 weekly sales files one by one and checks that each repeats the earlier weeks unchanged, keeps the same 599 rows in the same order, and adds exactly one new week.
What you see. "All 9 sales files are consistent".
Why it matters. Proves the files agree with each other, so using the last file (which holds every week) is safe.

Code (start)

```
print(D.check_weekly_files(cfg.paths.raw_dir, panel))
```

Output

```
All 9 sales files are consistent: same rows, same history, one extra week each.
```

Cell 11 · TEXT CELL

Text: reads the audit. Data is complete; 43% of weeks sell nothing; about 1 week in 10 is out of stock.

2 · Exploratory data analysis

Cell 12 · TEXT CELL

Heading: step 2, exploratory data analysis (EDA).

Cell 13 · CODE CELL

What it does. Turns the wide table (one column per week) into a long table (one row per store-product-week) with the product groups attached. Also defines hist_cols, which marks the 157 history weeks.
What you see. The first rows of the long table (98,835 rows in total).
Why it matters. A long table is easier for charts and group totals. hist_cols makes sure the 8 test weeks stay hidden.

Code (start)

```
long = D.to_long(panel)                          # one row per store-product-week
hist_cols = panel.sales.columns <= panel.week0   # the 157 history weeks; the 8 test weeks stay unseen
long.head()
```

Output: 1 table(s), shown in the notebook.

Cell 14 · TEXT CELL

Heading: 2.1, line graph of the whole network over time.

Cell 15 · CODE CELL

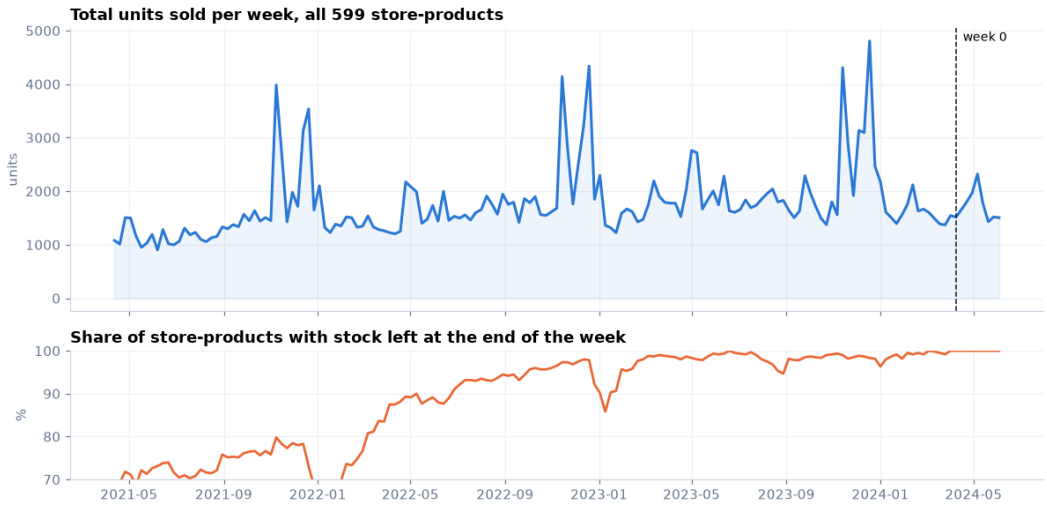
What it does. Adds up the sales of all 599 store-products for each week, and the share of store-products that had stock left. Draws both as line graphs.
What you see. Top: total units per week with yearly peaks. Bottom: the in-stock share, which dips in some periods. Mean about 1,761 units a week.
Why it matters. Shows the big picture: a stable level, a strong Christmas peak, and periods with many empty shelves.

Code (start)

```
total = panel.sales.sum(axis=0)                    # units sold by all store-products, per week
instock_share = panel.in_stock.mean(axis=0)        # share of store-products with stock left, per week
plots.network_over_time(total, instock_share, panel.week0)
print(f"Average week: {total.mean():,.0f} units")
```

Output

```
Average week: 1,761 units
```



Cell 16 · TEXT CELL

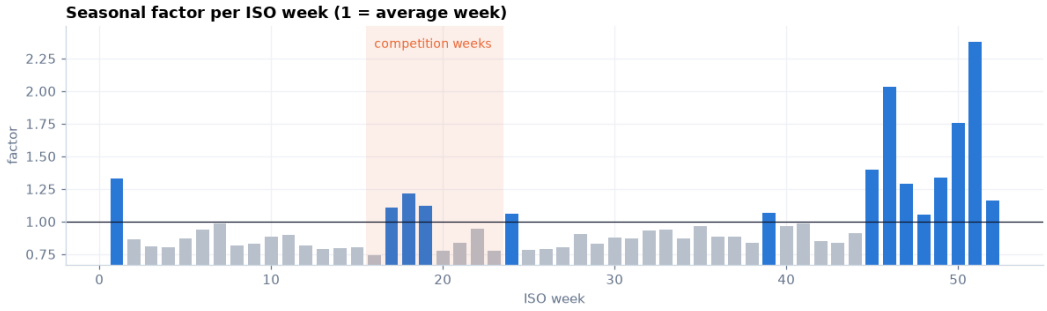
Text: sales are stable with a yearly rhythm; availability is not, so the in-stock flag matters.

Cell 17 · CODE CELL

What it does. Computes the seasonal factor of each week of the year: the average sales in that week ÷ the average week. This is exactly how the official benchmark measures seasonality.
What you see. A bar chart with 52 bars; weeks above 1 sell more than average; the test weeks (16-23) are shaded.
Why it matters. Every seasonal model later uses these factors.

Code (start)

```
season = M.season_factors(hist.upto(t0))           # average sales of each ISO week / average week
plots.seasonal_factors(season)
```



Cell 18 · TEXT CELL

Heading: 2.3, histograms (how much does one store-product sell?).

Cell 19 · CODE CELL

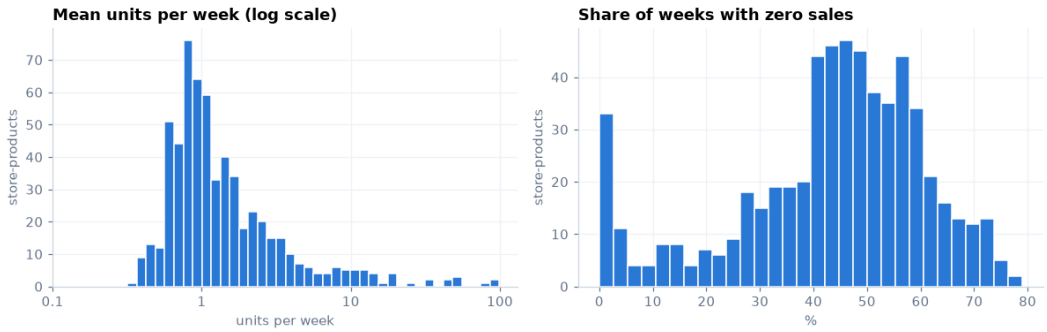
What it does. Computes, for each store-product, its average weekly sales and its share of zero weeks, and draws two histograms.
What you see. Most store-products sell about 1 unit a week; 75% sell under 2 a week.
Why it matters. Shows that demand is small and slow, so the models must handle many zeros.

Code (start)

```
sales_h = panel.sales.loc[:, hist_cols]            # history weeks only
mean_week = sales_h.mean(axis=1)                   # average units per week, per store-product
zero_share = (sales_h == 0).mean(axis=1)           # share of weeks with no sale
plots.sales_histograms(mean_week, zero_share)
print(f"Median store-product: {mean_week.median():.1f} units/week; {(mean_week < 2).mean():.0%} sell under 2 a
week")
```

Output

```
Median store-product: 1.1 units/week; 75% sell under 2 a week
```



Cell 20 · TEXT CELL

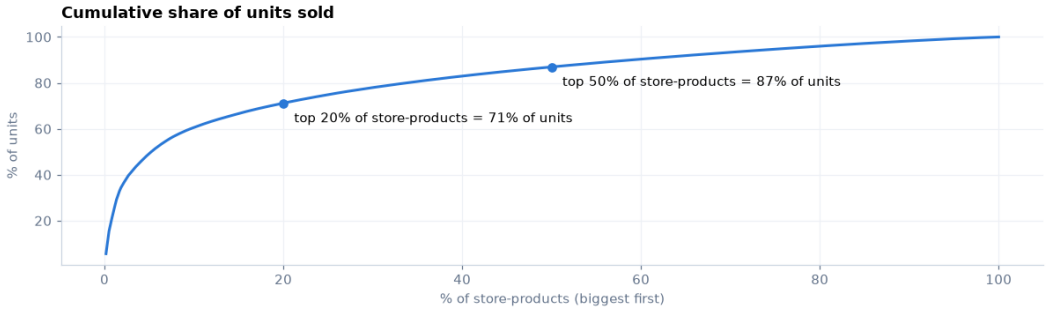
Text: demand is very skewed; errors measured in units are dominated by the big sellers. Heading 2.4: Pareto.

Cell 21 · CODE CELL

What it does. Sorts store-products from biggest to smallest seller and draws the running total of their share of sales (a Pareto curve).
What you see. A curve with two marked points, for example "top 20% of rows = X% of units".
Why it matters. Shows where the volume is: a small group of products makes most of the sales.

Code (start)

```
volume = sales_h.sum(axis=1).sort_values(ascending=False)    # total units per store-product, biggest first
plots.pareto(volume.cumsum() / volume.sum())
```



Cell 22 · TEXT CELL

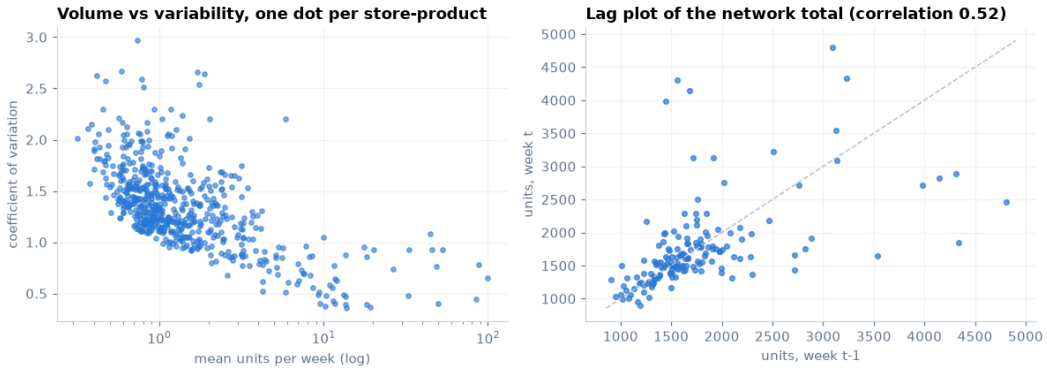
Heading: 2.5, scatter plots, and what they show.

Cell 23 · CODE CELL

What it does. Left: each store-product as a dot, average sales against variability (coefficient of variation). Right: the network total, this week against last week (a lag plot).
What you see. Slow sellers are the most variable. The lag-plot dots lie near a diagonal line (high correlation).
Why it matters. The lag plot shows that this week depends on last week, which is what AR (autoregressive) models use.

Code (start)

```
total_h = total.loc[:panel.week0].to_numpy()                # network total, history weeks
cv = sales_h.std(axis=1) / mean_week                        # coefficient of variation per store-product
plots.scatter_plots(mean_week, cv, total_h)
```



Cell 24 · TEXT CELL

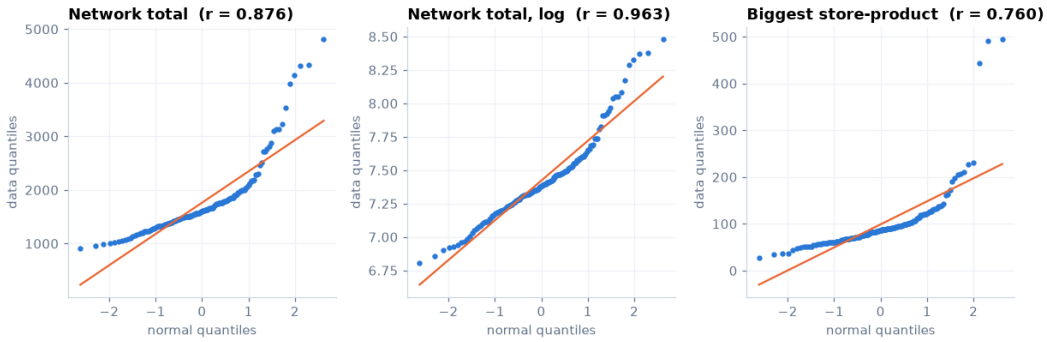
Text: reads the scatter plots. Heading 2.6: QQ plots, and what a QQ plot is.

Cell 25 · CODE CELL

What it does. Draws QQ plots for the network total, its log, and the biggest single store-product. Each plot compares the data with a perfect normal (bell-curve) distribution.
What you see. Points follow the line in the middle but bend away at the top (heavy tail).
Why it matters. Sales are not normally distributed. That decides later choices, such as using a count distribution for risk.

Code (start)

```
biggest = int(sales_h.sum(axis=1).to_numpy().argmax())       # the store-product with the most sales
plots.qq_plots({"Network total": total_h, "Network total, log": np.log(total_h),
                "Biggest store-product": sales_h.iloc[biggest].to_numpy()})
```



Cell 26 · TEXT CELL

Text: the total is close to normal in the middle with a heavy upper tail; single products are far from normal. Heading 2.7.

Cell 27 · CODE CELL

What it does. Computes descriptive statistics (mean, standard deviation, quartiles, CV, skewness, kurtosis, share of zero weeks, Jarque-Bera p-value) for the total, the biggest seller and the median product, then summarises the same statistics across all 599.
What you see. Two tables of numbers.
Why it matters. Puts numbers on what the charts showed: right-skewed, heavy-tailed, many zeros, not normal.

Code (start)

```
desc = TS.describe({"Network total": total_h, "Biggest store-product": sales_h.iloc[biggest].to_numpy(),
                    "Median of all store-products": sales_h.median(axis=0).to_numpy()})
display(desc.T.round(2))
per_row = TS.describe({i: sales_h.iloc[i].to_numpy() for i in range(hist.n)})   # the same for each store-produ
ct
per_row[["mean", "CV", "skewness", "excess kurtosis", "zero weeks %"]].astype(float).describe().round(2)
```

Output: 2 table(s), shown in the notebook.

Cell 28 · TEXT CELL

Text: reads the statistics. Heading 2.8: stockouts and product groups.

Cell 29 · CODE CELL

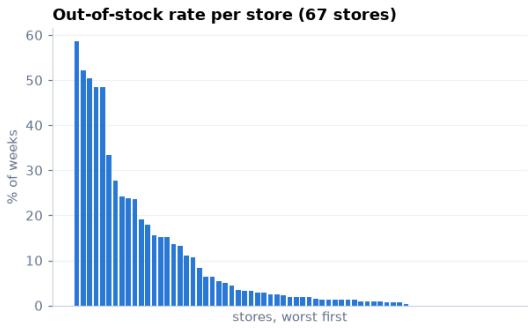
What it does. Computes the out-of-stock rate of each of the 67 stores and total sales per department group, and charts both.
What you see. Stores range from 0% to 59% of weeks out of stock; one chart of units per department group.
Why it matters. Shows that empty shelves are a real and uneven problem across stores.

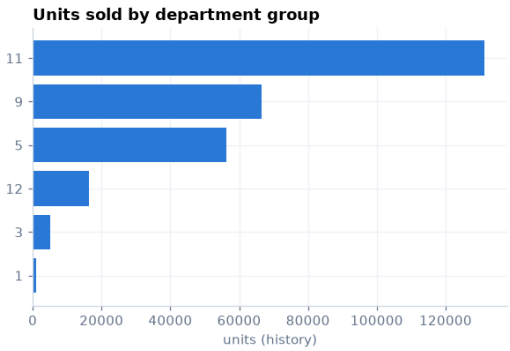
Code (start)

```
oos = 1 - panel.in_stock.loc[:, hist_cols].mean(axis=1)                    # out-of-stock rate per store-pro
duct
store_oos = oos.groupby(level="Store").mean().sort_values(ascending=False)     # ... per store
dept_sales = long[long.phase == "history"].groupby("DepartmentGroup")["sales"].sum().sort_values()
plots.stockouts_and_groups(store_oos, dept_sales)
print(f"Out-of-stock rate per store: from {store_oos.min():.0%} to {store_oos.max():.0%}")
```

Output

```
Out-of-stock rate per store: from 0% to 59%
```





Cell 30 · TEXT CELL

Text: what the EDA teaches us and how it shapes the models (slow demand, shared season, censoring, shared patterns).

3 · Dealing with missing values

Cell 31 · TEXT CELL

Heading: step 3, missing values. There are no empty cells, but two kinds of weeks are hidden missing values.

Cell 32 · TEXT CELL

Heading 3.1 and text: zeros before a product's first sale mean it was not on sale yet, not that nobody wanted it.

Cell 33 · CODE CELL

What it does. Finds each store-product's first week with a sale (its launch) and marks every earlier week. Draws a histogram of launch weeks.
What you see. 38% sell from the first week; 371 store-products were launched later.
Why it matters. Those early zeros would wrongly drag forecasts down, so they are treated as missing.

Code (start)

```
launched = D.launch_mask(panel.sales)                    # True from the first week with a sale
first_sale = launched.to_numpy().argmax(axis=1)          # week number of the first sale
print(f"Selling from the first week: {(first_sale == 0).mean():.0%} · launched later: {(first_sale > 0).sum()}"
)
plots.launch_histogram(first_sale)
```

Output

```
Selling from the first week: 38% · launched later: 371
```



Cell 34 · TEXT CELL

Heading 3.2 and text: out-of-stock weeks are censored (sales understate demand), so they are set aside.

Cell 35 · CODE CELL

What it does. Builds demand: the sales table with out-of-stock weeks and pre-launch weeks replaced by NaN (empty). Then measures how much is set aside and what out-of-stock weeks look like.
What you see. 4.3% of weeks set aside; 100% of out-of-stock weeks have zero sales.
Why it matters. This is the key cleaning step. An empty shelf means nothing could sell, so that zero says nothing about demand.

Code (start)

```
demand = D.clean_demand(panel.sales, panel.in_stock)      # out-of-stock and pre-launch weeks become NaN (missi
ng)

in_stock_h, launched_h = panel.in_stock.loc[:, hist_cols], launched.loc[:, hist_cols]
censored = ~in_stock_h & launched_h                       # out-of-stock weeks after launch
print(f"Weeks set aside as censored: {censored.to_numpy().mean():.1%}")
print(f"Out-of-stock weeks with zero sales: {(sales_h[censored].stack().dropna() == 0).mean():.0%}")
```

Output

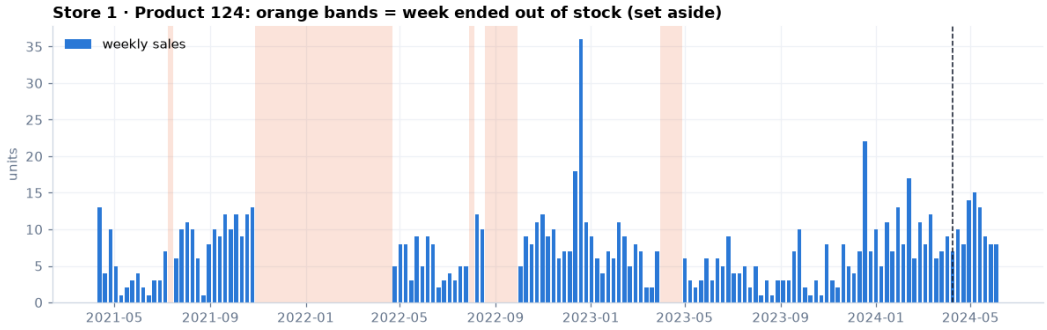
```
Weeks set aside as censored: 4.3%
Out-of-stock weeks with zero sales: 100%
```

Cell 36 · CODE CELL

What it does. Picks a busy store-product (more than 3 units a week) that often ran out (over 20% of weeks), and charts its sales with orange bands on out-of-stock weeks.
What you see. A bar chart with orange bands.
Why it matters. Makes the problem visible: the bands fall exactly on the weeks with zero sales.

Code (start)

```
oos_rate = (~in_stock_h).mean(axis=1)
example = oos_rate[(oos_rate > 0.2) & (mean_week > 3)].index[0]    # a busy store-product that often ran out
plots.stockout_example(panel.sales.loc[example], panel.in_stock.loc[example], panel.week0,
                       f"Store {example[0]} · Product {example[1]}")
```



Cell 37 · TEXT CELL

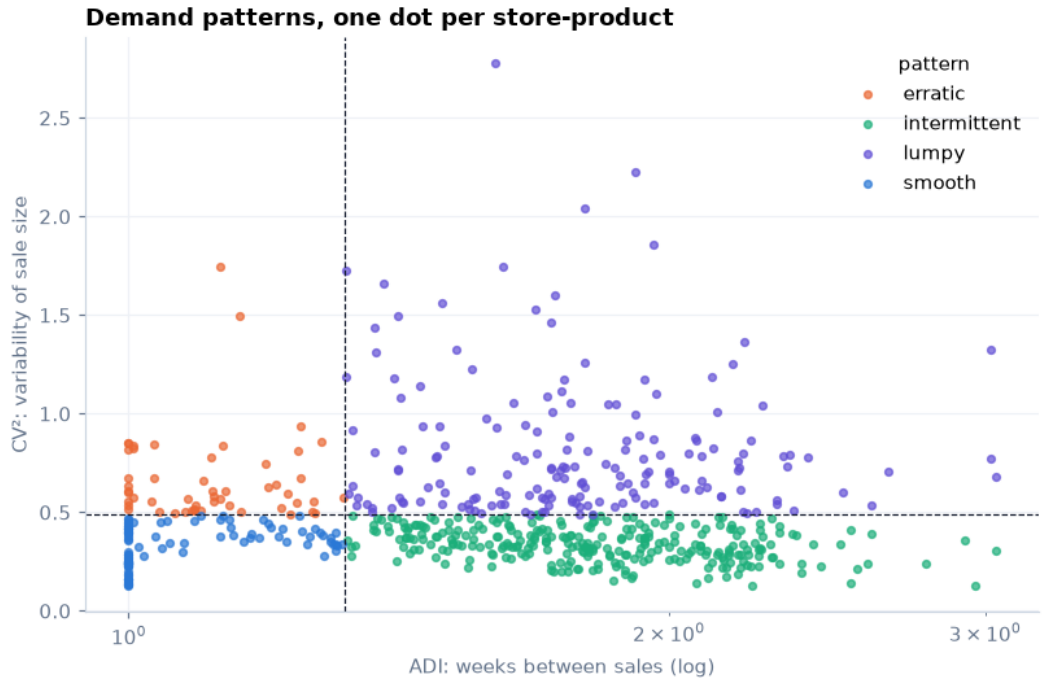
Text: every flagged week had zero sales. Then 3.3, a table of how each method treats a missing week, and 3.4, demand patterns (ADI and CV²).

Cell 38 · CODE CELL

What it does. Classifies every store-product with two numbers: ADI (average weeks between sales) and CV² (how much sale sizes vary). The standard cut-offs give 4 patterns: smooth, erratic, intermittent, lumpy.
What you see. A count table and a scatter plot with four coloured groups.
Why it matters. Each pattern suits different methods; Croston and TSB were designed for the intermittent and lumpy ones.

Code (start)

```
classes = D.classify_demand(demand.loc[:, hist_cols])     # ADI, CV² and pattern per store-product
display(classes["pattern"].value_counts())
plots.pattern_map(classes, D.ADI_CUT, D.CV2_CUT)
```



Cell 39 · TEXT CELL

Heading 3.5 and a table: how the 165 weeks are split (training, backtest, tuning, final test). No peeking at the future.

Cell 40 · CODE CELL

What it does. Defines the 30 backtest dates (origins): the first 12 for measuring forecast spread, the last 18 for accuracy and tuning. Draws a timeline of how the weeks are used.
What you see. A coloured bar timeline.
Why it matters. Keeps the test honest: the 8 competition weeks are never used to tune anything.

Code (start)

```
origins = list(range(t0 - 3 - cfg.forecast.backtest_origins + 1, t0 - 2))   # 30 past forecast dates
disp_origins = origins[:cfg.forecast.dispersion_origins]                    # first 12: error spread only
tune_origins = origins[cfg.forecast.dispersion_origins:]                    # last 18: accuracy and policy tuni
ng
plots.time_split(panel.sales.shape[1], t0, disp_origins, tune_origins, panel.dates)
```



4 · Stationarity: the ADF test

Cell 41 · TEXT CELL

Heading: step 4, stationarity. Explains the ADF test (H0: not stationary, p < 0.05 means stationary) and KPSS (the opposite), and the two series studied.

Cell 42 · CODE CELL

What it does. Picks the representative store-product (high volume, on sale from week 1, rarely out of stock). Builds y_net (the network total) and y_row (that product's clean demand, with gaps filled for the tests). Runs ADF and KPSS on both.
What you see. Store 61 · Product 124 (about 99 a week). A table: ADF says stationary for both.
Why it matters. ARIMA-type models need a stationary series, so this is checked first.

Code (start)

```
# The representative store-product: biggest seller among those sold from week 1, rarely out of stock, steady pa
ttern
steady = (first_sale == 0) & (oos_rate.to_numpy() < .05) & (classes["pattern"].to_numpy() == "smooth")
ts_row = int(np.flatnonzero(steady)[sales_h[steady].sum(axis=1).to_numpy().argmax()])
ts_name = f"Store {panel.index[ts_row][0]} · Product {panel.index[ts_row][1]}"

y_net = total_h                                          # series 1: the network total
y_row = demand.iloc[ts_row, :t0 + 1].to_numpy()          # series 2: clean demand of one store-product
… (4 more lines)
```

Output

```
Representative store-product: Store 61 · Product 124
```

Cell 43 · CODE CELL

What it does. Runs the ADF test on every one of the 599 store-products (clean weeks, gaps filled).
What you see. 94% stationary, 6% not.
Why it matters. Shows the result holds for almost every product, not just the two examples.

Code (start)

```
adf_all = TS.adf_all_rows(demand.loc[:, hist_cols])        # ADF p-value of every store-product
print(f"ADF on {len(adf_all)} store-products: {np.mean(adf_all < .05):.0%} stationary")
```

Output

```
ADF on 599 store-products: 94% stationary
```

5 · Transformations

Cell 44 · TEXT CELL

Text: ADF and KPSS readings. Heading: step 5, transformations (log, first difference, seasonal difference).

Cell 45 · CODE CELL

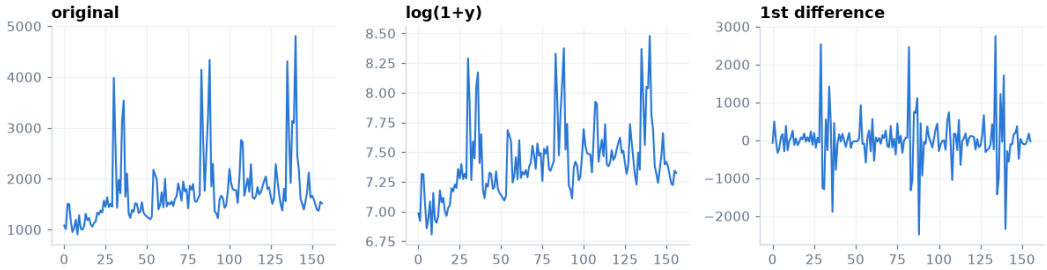
What it does. Applies each transformation to the network total, re-tests ADF and KPSS, and chooses d (how many times to difference) with the ADF rule. Plots three versions.
What you see. A results table and "d = 0" for both series (no differencing needed).
Why it matters. Proves the original series can be used as it is, so forecasts stay in real units.

Code (start)

```
tr = TS.transformations(y_net, 52)                          # original, log, 1st difference, seasonal difference
tr_tab = TS.stationarity(tr)
display(tr_tab[["ADF p-value", "ADF says", "KPSS p-value", "KPSS says", "verdict"]])
d_net, d_row = TS.choose_d(y_net), TS.choose_d(y_row_filled)   # difference only while ADF finds a unit root
print(f"Differencing chosen: network total d = {d_net}, {ts_name} d = {d_row}")
plots.transformations(tr)
```

Output

```
Differencing chosen: network total d = 0, Store 61 · Product 124 d = 0
```



6 · Seasonality: STL decomposition

Cell 46 · TEXT CELL

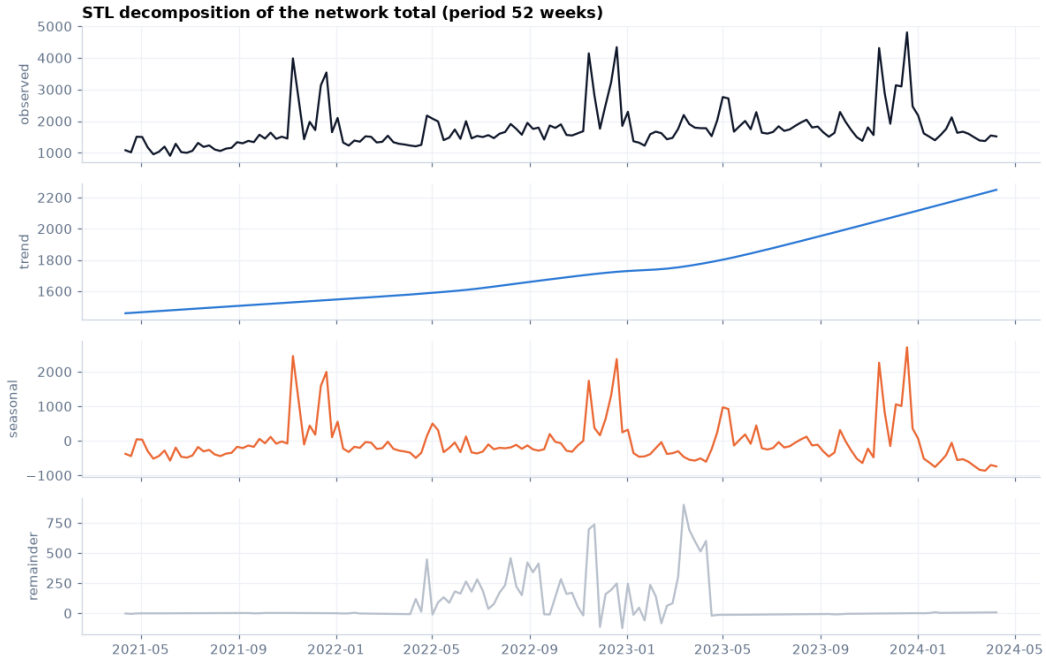
Text: why d = 0 is kept. Heading: step 6, STL decomposition and what trend and season strength mean.

Cell 47 · CODE CELL

What it does. Splits the network total into trend, seasonal and remainder with STL (52-week season), plots the four parts, and computes trend and seasonal strength for both series.
What you see. A four-panel chart; seasonal strength 0.93 for the total.
Why it matters. The data is very seasonal (Christmas), so a seasonal part is needed in the models.

Code (start)

```
dec = TS.decompose(y_net, dates_h, 52)                      # trend + seasonal + remainder
plots.decomposition(dec)
s_net = TS.strength(dec)
s_row = TS.strength(TS.decompose(y_row_filled, dates_h, 52))
pd.DataFrame({"Network total": s_net, ts_name: s_row}).T.round(2)
```



7 · ACF and PACF

Cell 48 · TEXT CELL

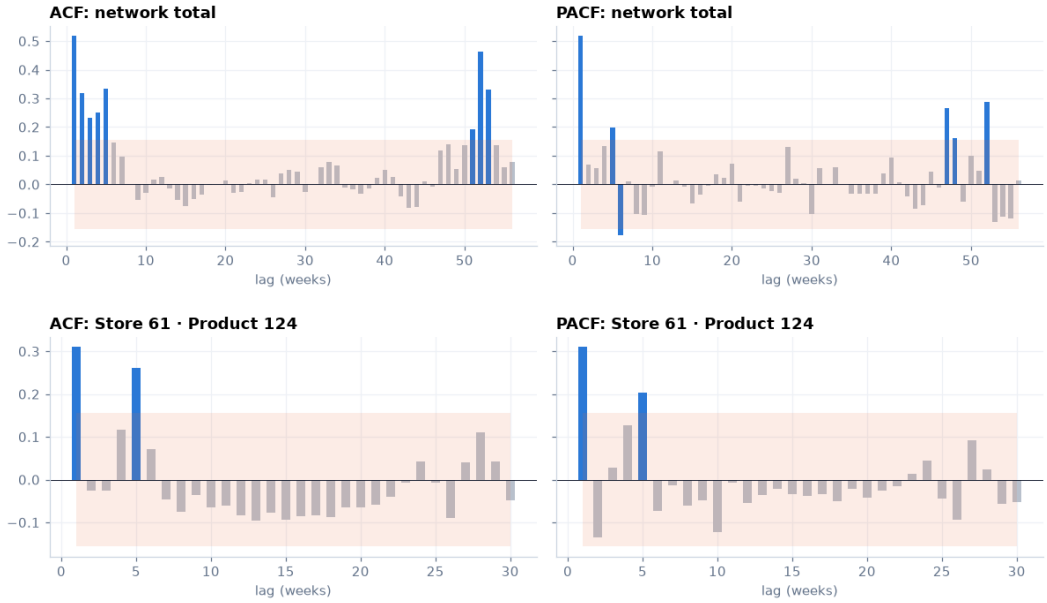
Text: reads the decomposition. Heading: step 7, what ACF and PACF are.

Cell 49 · CODE CELL

What it does. Computes the ACF and PACF of the network total (56 lags) and of the representative product (30 lags), and draws them with the 95% band. No differencing is applied because d = 0.
What you see. Two pairs of bar charts; blue bars cross the band (significant), grey bars do not.
Why it matters. These plots show which past weeks matter, which guides the choice of model.

Code (start)

```
ap_net = TS.acf_pacf(y_net, 56)                    # d = 0, so the series is used as it is
ap_row = TS.acf_pacf(y_row_filled, 30)
plots.acf_pacf(ap_net, "network total")
plots.acf_pacf(ap_row, ts_name)
```



8 · Identify the model: AR, MA, ARMA, ARIMA or SARIMA?

Cell 50 · TEXT CELL

Heading: step 8, model identification, with the textbook rules table (AR, MA, ARMA, ARIMA, SARIMA).

Cell 51 · CODE CELL

What it does. Reads the ACF/PACF automatically with those rules and suggests a model family and orders (p, d, q).
What you see. Network total: AR with a seasonal spike, so SARIMA. The product: ARMA.
Why it matters. Turns the plots into a starting model, which step 10 then checks with AIC/BIC.

Code (start)

```
id_net = TS.identify(ap_net, d_net, seasonal_lag=52)       # reads the plots with the Box-Jenkins rules
id_row = TS.identify(ap_row, d_row)
pd.DataFrame({"Network total": id_net, ts_name: id_row}).astype(str)
```

Output: 1 table(s), shown in the notebook.

9 · Identify the lags

Cell 52 · TEXT CELL

Text: why the total looks like a seasonal AR process. Heading: step 9, lags.

Cell 53 · CODE CELL

What it does. Lists the significant lags of the ACF and PACF for both series.
What you see. Network total: lags 1-5 and around 52. The product: lags 1 and 5.
Why it matters. Recent weeks and the same week last year carry the information; this also chose LightGBM's inputs.

Code (start)

```
print("Network total:", TS.significant_lags(ap_net))
print(ts_name + ":", TS.significant_lags(ap_row))
```

Output

```
Network total: {'acf': [1, 2, 3, 4, 5, 51, 52, 53], 'pacf': [1, 5, 6, 47, 48, 52]}
Store 61 · Product 124: {'acf': [1, 5], 'pacf': [1, 5]}
```

10 · AIC and BIC: choose the best order

Cell 54 · TEXT CELL

Text: what the lags mean. Heading: step 10, AIC and BIC formulas and the grid to compare.

Cell 55 · CODE CELL

What it does. Fits 27 candidate models to the network total (ARMA, ARIMA with d = 1, and SARIMA with a 52-week seasonal term) and ranks them by AIC; also shows the BIC winner.
What you see. Best by AIC: SARIMA(2,0,0)(1,0,0)[52]; best by BIC: SARIMA(1,0,0)(1,0,0)[52]. A table of the top 10.
Why it matters. Picks the best model objectively, balancing fit against complexity.

Code (start)

```
# 27 candidates: ARMA and SARIMA (seasonal AR at lag 52) with d = 0, and ARIMA with d = 1
grid_all = pd.concat([TS.aic_bic_grid(y_net, d_net, seasonal_orders=[(0, 0, 0, 0), (1, 0, 0, 52)]),
                      TS.aic_bic_grid(y_net, 1)]).sort_values("AIC").reset_index(drop=True)
grid_all["ΔAIC"] = grid_all["AIC"] - grid_all["AIC"].min()
best_net = grid_all.iloc[0]                                  # lowest AIC
best_bic = grid_all.sort_values("BIC").iloc[0]               # lowest BIC
print("Best by AIC:", best_net["model"], "· best by BIC:", best_bic["model"])
… (1 more lines)
```

Output

```
Best by AIC: SARIMA(2, 0, 0)x(1, 0, 0)[52] · best by BIC: SARIMA(1, 0, 0)x(1, 0, 0)[52]
```

Cell 56 · CODE CELL

What it does. Does the same AIC/BIC comparison for the representative product. Missing weeks stay missing; the Kalman filter skips them.
What you see. Best: MA(1) by both AIC and BIC.
Why it matters. Shows the method works for a single product too.

Code (start)

```
grid_row = TS.aic_bic_grid(y_row, d_row)                     # missing weeks stay missing: the Kalman filter skip
s them
print(ts_name, "· best by AIC:", grid_row.iloc[0]["model"])
grid_row[["model", "AIC", "BIC", "ΔAIC"]].head(6).round(1)
```

Output

```
Store 61 · Product 124 · best by AIC: MA(0, 0, 1)
```

Cell 57 · TEXT CELL

Text: the same recipe will now run automatically on all 599 store-products (season removed first).

Cell 58 · CODE CELL

What it does. Fits an ARIMA to every store-product in parallel: ADF chooses d, AIC chooses p and q, and products with fewer than 26 clean weeks get a simple mean.
What you see. Counts per family (ARMA 229, mean only 166, AR 119, MA 53, ARIMA 32) and the most common orders.
Why it matters. This per-product ARIMA becomes one of the forecasting methods in step 12.

Code (start)

```
arima_rows = TS.RowARIMA().fit(hist.upto(t0))                # one ARIMA per store-product (runs in parallel, ~1
min)
arima_tab = arima_rows.table
display(arima_tab["family"].value_counts())
arima_tab["order"].astype(str).value_counts().head(8)        # most common (p, d, q)
```

Output: 2 table(s), shown in the notebook.

11 · Residual diagnostics: is what is left white noise? ARCH and GARCH

Cell 59 · TEXT CELL

Text: reads the result. Heading: step 11, residual tests (Ljung-Box, Jarque-Bera, ARCH-LM) and GARCH.

Cell 60 · CODE CELL

What it does. Fits the winning SARIMA to the network total, prints its coefficients, and runs the three tests on its residuals (what it could not explain).
What you see. A coefficient table (all significant) and test results: Ljung-Box borderline at lag 8, ARCH effect found (p = 0.027).
Why it matters. Checks whether the model left any pattern behind.

Code (start)

```
fit_net = TS.fit_arima(y_net, best_net["order"], best_net["seasonal"])
burn = int(fit_net.loglikelihood_burn) + 1                # the first residuals have no past yet: skip them
resid_net = np.asarray(fit_net.resid)[burn:]
rt_net = TS.residual_tests(resid_net, n_params=len(fit_net.params) - 1)
print(fit_net.summary().tables[1])
pd.Series(rt_net, name=best_net["model"])
```

Output

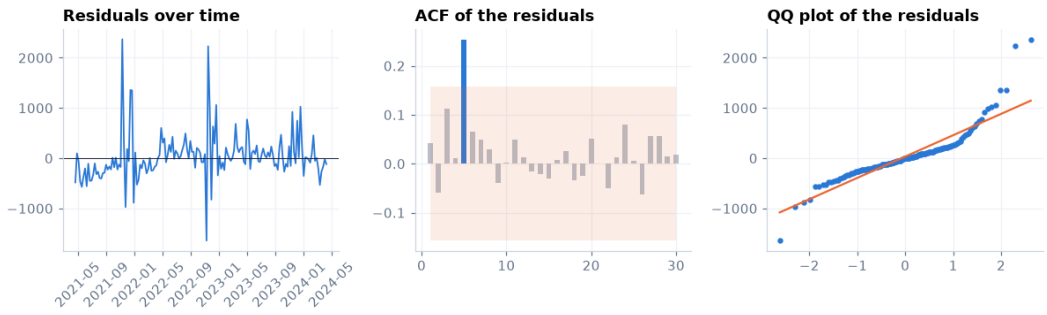
```
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
intercept    280.4228     65.020      4.313      0.000     152.985     407.860
ar.L1          0.3223      0.042      7.735      0.000       0.241       0.404
ar.L2          0.1519      0.049      3.085      0.002       0.055       0.248
ar.S.L52       0.6950      0.028     24.790      0.000       0.640       0.750
sigma2      1.482e+05   9607.202     15.427      0.000    1.29e+05    1.67e+05
==============================================================================
```

Cell 61 · CODE CELL

What it does. Draws three diagnostic charts of the residuals: over time, their ACF, and a QQ plot.
What you see. Residuals hover around 0; almost all ACF bars are inside the band; the QQ plot has heavy tails.
Why it matters. A visual check that matches the tests.

Code (start)

```
plots.residual_diagnostics(dates_h[burn:], resid_net, TS.acf_pacf(resid_net, 30))
```



Cell 62 · TEXT CELL

Text: reads the charts; the ARCH effect means calm and turbulent periods alternate, so GARCH is next.

Cell 63 · CODE CELL

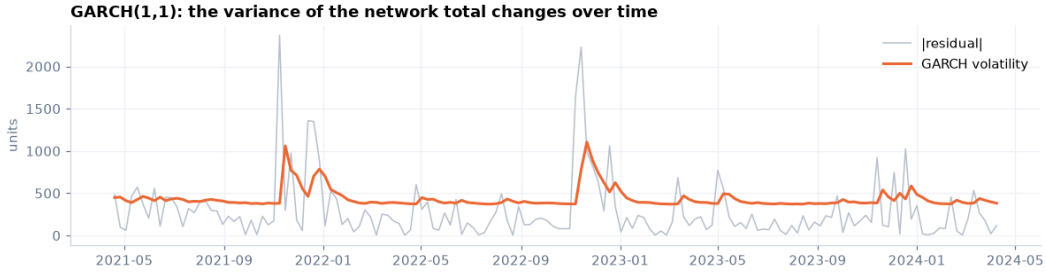
What it does. Fits GARCH(1,1) to the residuals, plots the changing volatility, and repeats the tests on the standardised residuals and their squares.
What you see. ω = 0.35, α = 0.18, β = 0.45. After GARCH, Ljung-Box passes and the ARCH effect is gone.
Why it matters. Confirms the leftover is white noise once the changing noise level is modelled.

Code (start)

```
garch_net, gscale = TS.fit_garch(resid_net)               # GARCH(1,1) on the residuals
print(garch_net.params.round(3))
std_net = np.asarray(garch_net.std_resid)                 # residual / its GARCH volatility
gtests = pd.DataFrame({"SARIMA residuals": rt_net, "after GARCH": TS.residual_tests(std_net),
                       "after GARCH, squared": TS.residual_tests(std_net ** 2)})
plots.garch_volatility(dates_h[burn:], resid_net, np.asarray(garch_net.conditional_volatility) * gscale)
gtests
```

Output

```
omega      0.35
alpha[1]   0.18
beta[1]    0.45
Name: params, dtype: float64
```



Cell 64 · TEXT CELL

Text: what the GARCH result means (normality still fails because of a few extreme weeks). Heading 11.2.

Cell 65 · CODE CELL

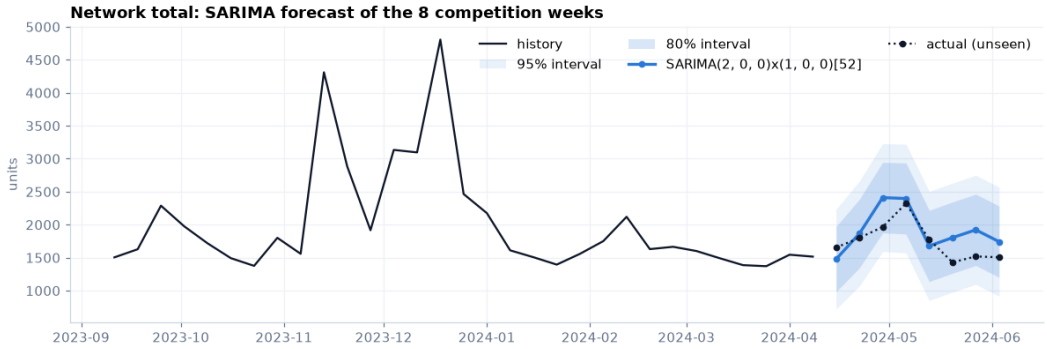
What it does. Uses the SARIMA fitted only on history to forecast the 8 competition weeks, with 80% and 95% ranges, and compares with what really sold and with "same week last year".
What you see. Error 14.2% vs 20.0%; all 8 real weeks fall inside the 80% range.
Why it matters. A real out-of-sample test: the model predicts weeks it never saw.

Code (start)

```
fc_net = fit_net.get_forecast(8)                          # fitted on history only, forecasts weeks 1-8
mean_net = np.asarray(fc_net.predicted_mean)
ci80, ci95 = np.asarray(fc_net.conf_int(alpha=.2)), np.asarray(fc_net.conf_int(alpha=.05))
actual_net = panel.sales.iloc[:, t0 + 1:t0 + 9].sum(axis=0).to_numpy()
last_year = total.iloc[t0 + 1 - 52:t0 + 9 - 52].to_numpy()                # "same week last year"
fc_dates = panel.dates[t0 + 1:t0 + 9]
plots.holdout_forecast(total.iloc[t0 - 30:t0 + 1], fc_dates, mean_net, ci80, ci95, actual_net, best_net["model"
])
… (5 more lines)
```

Output

```
Error (MAPE): SARIMA 14.2% · same week last year 20.0%
Real weeks inside the 80% range: 100%
```



Cell 66 · TEXT CELL

Heading 11.3: the same tests on every store-product.

Cell 67 · CODE CELL

What it does. Summarises the residual tests of the 599 ARIMA models, draws p-value histograms, then fits GARCH(1,1) to every product with an ARCH effect and re-tests.
What you see. 91% white noise; 10% ARCH effect; GARCH fixes 85% of those.
Why it matters. Shows the classical models are well specified across the whole chain.

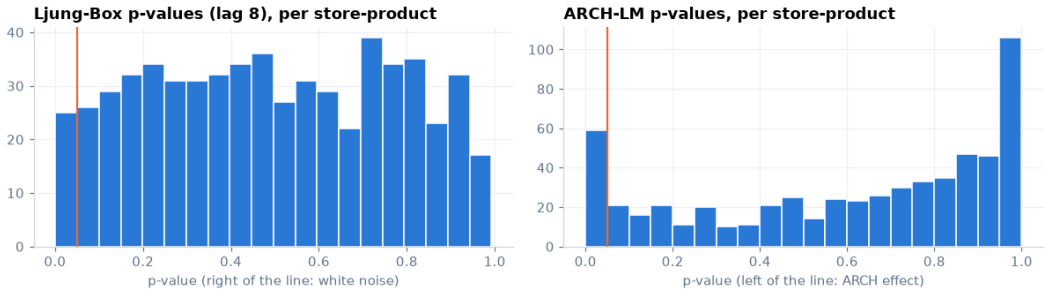
Code (start)

```
fitted = arima_tab.dropna(subset=["aic"])
wn = fitted["white noise"].astype(bool).mean()             # share passing Ljung-Box
arch_share = fitted["ARCH effect"].astype(bool).mean()     # share with volatility clustering
print(f"White-noise residuals: {wn:.0%} · ARCH effect: {arch_share:.0%}")
plots.pvalue_histograms(fitted["Ljung-Box p (lag 8)"], fitted["ARCH-LM p"])

season_h = M.season_of(M.season_factors(hist.upto(t0)), dates_h)
… (3 more lines)
```

Output

```
White-noise residuals: 91% · ARCH effect: 10%
GARCH(1,1) on the 59 store-products with an ARCH effect: clustering removed in 85% of them
```



Cell 68 · TEXT CELL

Text: summary of steps 4 to 11.

12 · Forecasting models

Cell 69 · TEXT CELL

Heading: step 12, forecasting methods. Why three weeks ahead, and a table of all 10 methods with their idea.

Cell 70 · TEXT CELL

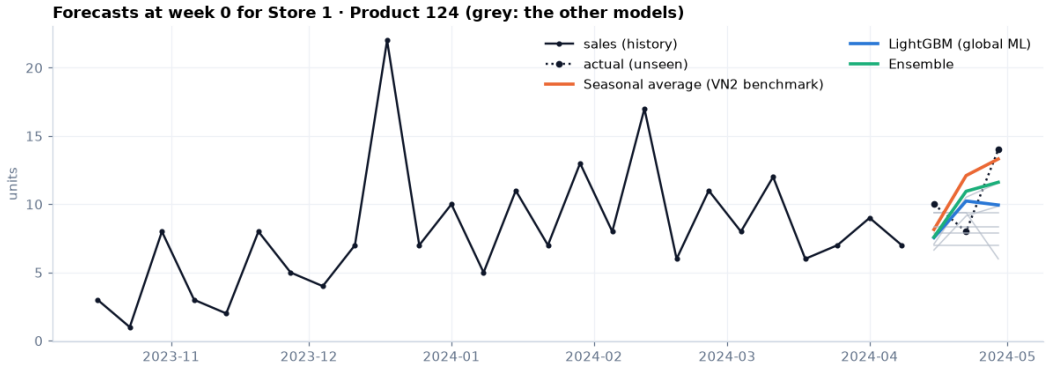
Text: LightGBM's input features and its Tweedie loss, and why.

Cell 71 · CODE CELL

What it does. Trains LightGBM once on all history, makes every method forecast the next 3 weeks for one example product, and plots them against the history and what really sold.
What you see. A chart with three highlighted methods and the others in grey.
Why it matters. Lets you see the methods side by side before measuring them properly.

Code (start)

```
lgbm_demo = M.GlobalLGBM(3, seed=SEED).fit(hist.upto(t0))  # one model trained on all store-products
demo = M.forecast_all(hist.upto(t0), 3, cfg.forecast.models, lgbm_demo, cfg.forecast.ensemble_of, arima_rows)
row = panel.index.get_loc(example)                         # the store-product shown in step 3
plots.model_forecasts(panel.sales.iloc[row, t0 - 25:t0 + 1], panel.sales.iloc[row, t0 + 1:t0 + 4],
                      {m: f[row] for m, f in demo.items()}, M.MODEL_LABELS,
                      f"Forecasts at week 0 for Store {example[0]} · Product {example[1]}")
```



13 · Backtest: which model forecasts best?

Cell 72 · TEXT CELL

Heading: step 13, the backtest and the four error measures (MAE, Bias, WAPE, Score).

Cell 73 · CODE CELL

What it does. The slowest cell (about 6 minutes). At each of 30 past dates, every method forecasts 3 weeks using only earlier data (LightGBM refitted every 4 dates, ARIMA every 8). All forecasts are compared with what sold.
What you see. The leaderboard table: MAE, RMSE, Bias, WAPE and Score for every method.
Why it matters. A fair test: no method ever sees the weeks it predicts.

Code (start)

```
# At each of the 30 dates, every model forecasts 3 weeks using only earlier data (about 6 minutes)
bt, bt_info = E.backtest(hist, origins, cfg.forecast.models, cfg.forecast.horizon, cfg.forecast.ensemble_of,
                         cfg.forecast.lgbm_retrain_every, SEED, arima_refit_every=cfg.forecast.arima_refit_ever
y)
lb = E.leaderboard(bt)                                     # MAE, RMSE, Bias, WAPE and Score per model
lb.insert(1, "label", lb["model"].map(M.MODEL_LABELS))
lb
```

Output: 1 table(s), shown in the notebook.

Cell 74 · CODE CELL

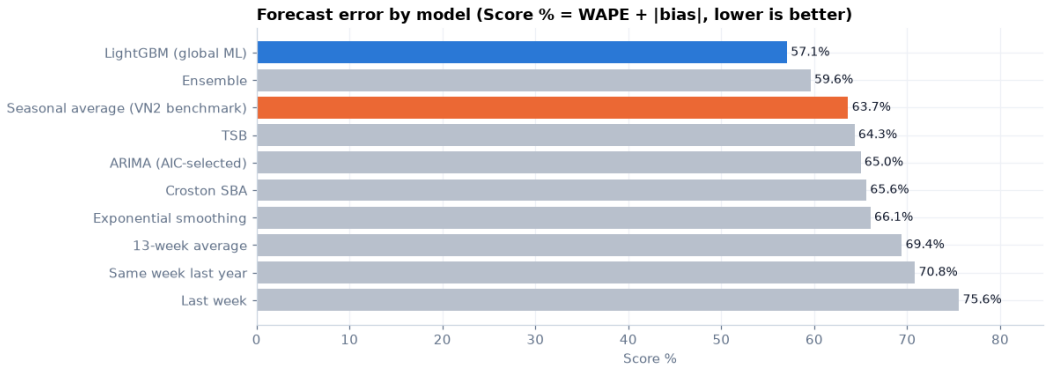
What it does. Draws the leaderboard as a bar chart.
What you see. LightGBM best (57.1%); the official benchmark forecast 63.7%; "last week" worst.
Why it matters. Shows clearly which forecast is most accurate.

Code (start)

```
plots.leaderboard(lb)
best_fc = lb.iloc[0]["model"]
bench_score = lb.set_index("model").loc["seasonal_ma", "Score %"]
print(f"Most accurate: {M.MODEL_LABELS[best_fc]} ({lb.iloc[0]['Score %']:.1f}%) · benchmark forecast {bench_sco
re:.1f}…
```

Output

```
Most accurate: LightGBM (global ML) (57.1%) · benchmark forecast 63.7%
```



Cell 75 · TEXT CELL

Text: next, accuracy by forecast horizon and by demand pattern.

Cell 76 · CODE CELL

What it does. Two tables: error 1, 2 and 3 weeks ahead, and error per demand pattern.
What you see. Further ahead is a little harder; intermittent and lumpy products are much harder.
Why it matters. Shows where forecasting struggles.

Code (start)

```
lb_h = E.leaderboard(bt, ["h"])                          # by weeks ahead
display(lb_h.pivot(index="model", columns="h", values="Score %").round(1))
bt_pat = bt.merge(classes[["pattern"]].reset_index(drop=True).rename_axis("row").reset_index(), on="row")
lb_p = E.leaderboard(bt_pat, ["pattern"])                # by demand pattern
lb_p.pivot(index="model", columns="pattern", values="Score %").round(1)
```

Output: 2 table(s), shown in the notebook.

Cell 77 · TEXT CELL

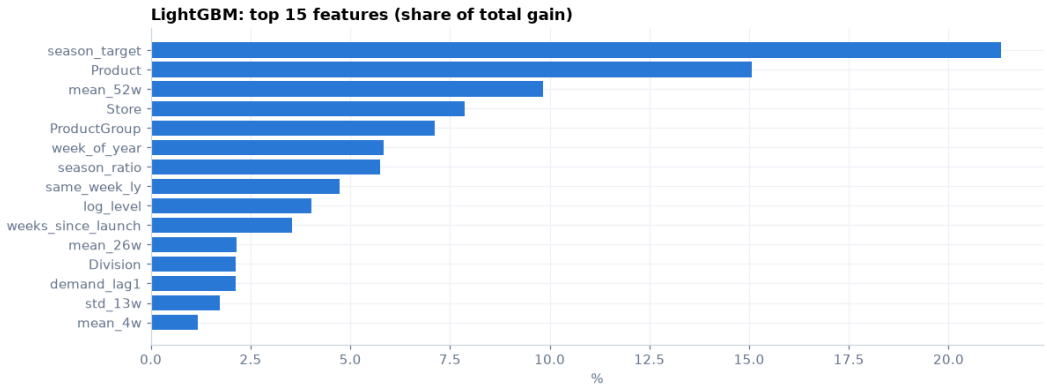
Text: reads the tables. Then: what drives LightGBM?

Cell 78 · CODE CELL

What it does. Charts LightGBM's 15 most important inputs (share of total gain).
What you see. Recent averages and lags lead; season and product groups follow.
Why it matters. Explains the machine-learning model in plain terms.

Code (start)

```
imp = bt_info["lgbm"].importance()                       # share of total gain per feature
plots.importance(imp)
```



Cell 79 · TEXT CELL

Text and heading 13.1: Ljung-Box on the real forecast errors of the backtest.

Cell 80 · CODE CELL

What it does. For every method and store-product, tests whether the 1-week-ahead errors over 30 weeks are white noise (Ljung-Box, lag 4).
What you see. Most methods: 89-93% of products pass; "last week": 52%.
Why it matters. A good forecast leaves no pattern in its mistakes. A pass does not prove accuracy; the leaderboard does that.

Code (start)

```
whiteness = TS.error_whiteness(bt, cfg.forecast.models, lag=4)   # Ljung-Box on each store-product's errors
whiteness.round(2)
```

Output: 1 table(s), shown in the notebook.

Cell 81 · TEXT CELL

Text: how to read the result, and why accuracy and white noise together are the evidence.

14 · Uncertainty: how wrong can a forecast be?

Cell 82 · TEXT CELL

Heading: step 14, uncertainty. Demand is modelled as a negative binomial with mean = forecast and variance k × mean.

Cell 83 · CODE CELL

What it does. Estimates the spread k for every product and method from backtest errors, then checks calibration: does demand stay below the 50%, 80% and 95% marks about 50%, 80% and 95% of the time?
What you see. A table: for example LightGBM 51%, 85%, 98%.
Why it matters. Without the spread, orders would cover only the average and run out about half the time.

Code (start)

```
disp_early = E.dispersion(bt[bt.origin.isin(disp_origins)], hist.n)   # spread k from dates 1-12 (for tuning)
disp_all = E.dispersion(bt, hist.n)                                   # spread k from all 30 dates (for the tes
t)
calib = E.calibration(bt[bt.origin.isin(tune_origins)], disp_early, M.MODEL_LABELS)
calib.round(2)
```

Output: 1 table(s), shown in the notebook.

15 · The inventory simulator (official rules)

Cell 84 · TEXT CELL

Text: the ranges are slightly generous, and step 16 corrects for that. Heading: step 15, the simulator, with the weekly rules written as five lines.

Cell 85 · TEXT CELL

Text: before trusting the simulator, reproduce the official benchmark score of €4,334.0.

Cell 86 · CODE CELL

What it does. Plays the organiser's published benchmark orders through our simulator, then rebuilds the benchmark method and compares its orders. Stops with an error if anything differs.
What you see. Setup €913.8 · score €4,334.0 · identical orders: True □ simulator verified, and the benchmark week by week.
Why it matters. Proves our scores are directly comparable with the real competition leaderboard.

Code (start)

```
costs = I.Costs(cfg.competition.holding_cost, cfg.competition.shortage_cost)   # €0.20 and €1
initial = I.State.from_initial(panel.initial)                                  # the real starting stock
comp_demand = panel.sales.iloc[:, t0 + 1:t0 + 9].to_numpy()                    # demand of weeks 1-8

# 1. The organiser's published benchmark orders
bench_orders = [o.to_numpy() for o in D.load_benchmark_orders(cfg.paths.raw_dir)]
bench_weekly, _, _ = I.simulate(initial, comp_demand, bench_orders, costs)
… (11 more lines)
```

Output

```
Setup €913.8 · score €4,334.0 · identical orders: True    simulator verified
```

16 · The ordering policy

Cell 87 · TEXT CELL

Text: the check passed. Heading: step 16, the ordering policy in 3 steps; its code is newsvendor_orders in src/optichain/inventory.py.

Cell 88 · TEXT CELL

Text: tuning by simulation, replaying 18 past weeks for every method and service level.

Cell 89 · CODE CELL

What it does. Replays the 18 past weeks for every method at 12 service levels, and the benchmark's order-up-to rule (function tune_service_levels), then keeps the cheapest setting per method (best_settings).
What you see. A table of the best service level and its replay cost per method (about 2.5 minutes).
Why it matters. Chooses the safety level from real simulated costs, not from a formula.

Code (start)

```
grid = cfg.policy.service_grid
tuning = P.tune_service_levels(bt, tune_origins, hist.sales, disp_early, costs, cfg.forecast.models, grid,
                               samples=cfg.policy.samples, seed=SEED, warmup=cfg.policy.replay_warmup)
best_service = P.best_settings(tuning, "newsvendor")       # cheapest service level per model
best_cov = P.best_settings(tuning, "order_up_to")          # cheapest coverage per model (benchmark-style rule)
best_service.sort_values("cost")
```

Output: 1 table(s), shown in the notebook.

Cell 90 · CODE CELL

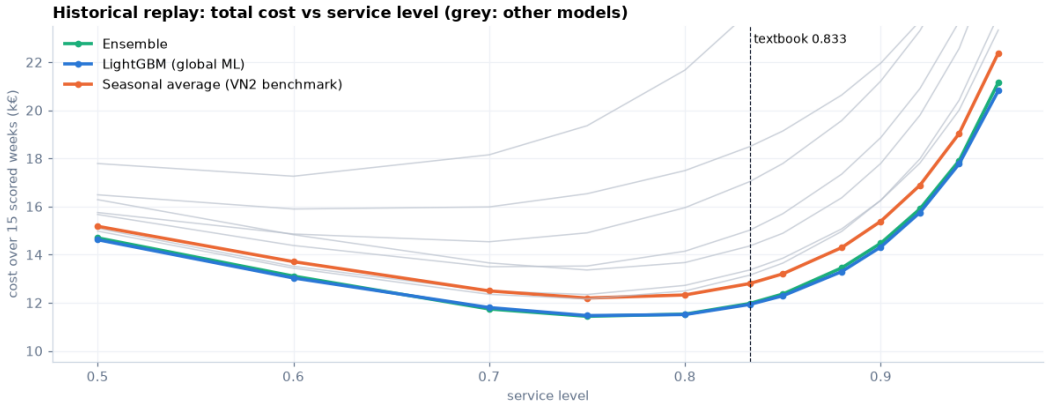
What it does. Draws cost against service level for every method and picks the overall winner.
What you see. U-shaped curves; the chosen plan is Ensemble at service level 0.75.
Why it matters. The bottom of the U is the best balance between empty shelves and leftover stock.

Code (start)

```
plots.tuning_curves(tuning[tuning.policy == "newsvendor"], M.MODEL_LABELS, costs.critical_ratio)
plan_model = best_service["cost"].idxmin()                # the model with the cheapest replay
plan_service = float(best_service.loc[plan_model, "param"])
print(f"Chosen plan: {M.MODEL_LABELS[plan_model]} at service level {plan_service}")
```

Output

```
Chosen plan: Ensemble at service level 0.75
```



17 · Final test: the eight VN2 weeks

Cell 91 · TEXT CELL

Text: better forecasts mean lower cost, and why 0.75 is below the textbook 0.833. Heading: step 17, the final test.

Cell 92 · CODE CELL

What it does. Makes forecasts for each of the 9 rounds (weekly, retraining LightGBM on the data known then) and measures accuracy on the competition weeks.
What you see. An accuracy table for the 8 test weeks (about 6 minutes to compute the forecasts).
Why it matters. Simulates exactly how the competition worked, week by week.

Code (start)

```
# Forecasts of every round, refitting on the data known then (about 6 minutes)
rounds = P.round_forecasts(hist, t0, cfg.forecast.models, cfg.forecast.ensemble_of, n_rounds=9, horizon=4, seed
=SEED)

# Accuracy on the competition weeks (1 to 3 weeks ahead, clean weeks only)
acc = rounds[(rounds.h <= 3) & (rounds.week <= 8)].copy()
acc["demand"] = demand.to_numpy()[acc["row"], t0 + acc["week"]]
lb_comp = E.leaderboard(acc)
… (2 more lines)
```

Output: 1 table(s), shown in the notebook.

Cell 93 · TEXT CELL

Text: an honest surprise. On these 8 weeks the simple benchmark forecast was the most accurate.

Cell 94 · CODE CELL

What it does. Plays the 8 competition weeks for every method with its service level tuned on the past, and also the order-up-to rule; records the score and the "hindsight best".
What you see. A table: Ensemble €3,626 is lowest; LightGBM €3,680; ARIMA €3,757.
Why it matters. The real test: what each forecast would have cost in the actual competition.

Code (start)

```
comp_scores, comp_grid = P.score_competition(initial, comp_demand, rounds, disp_all, costs, cfg.forecast.models
,
                                             M.MODEL_LABELS, best_service, best_cov, grid, cfg.policy.samples,
SEED)
comp_scores
```

Output: 1 table(s), shown in the notebook.

Cell 95 · CODE CELL

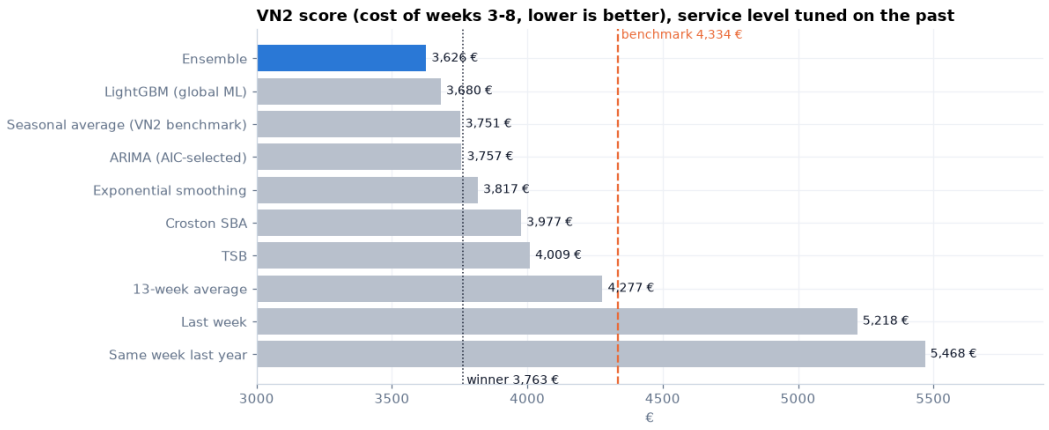
What it does. Charts every method's score against the official benchmark and the real winner.
What you see. Plan: Ensemble at 0.75, €3,626.4, 16.3% below the benchmark.
Why it matters. The headline result of the project.

Code (start)

```
plan_score = float(comp_scores.set_index("model").loc[plan_model, "VN2 score €"])
plots.competition_scores(comp_scores, plan_model, cfg.competition.benchmark_score, cfg.competition.winner_score
)
print(f"Plan: {M.MODEL_LABELS[plan_model]} @ {plan_service} → €{plan_score:,.1f} "
      f"({1 - plan_score / bench_official:.1%} below the benchmark)")
```

Output

```
Plan: Ensemble @ 0.75 → €3,626.4 (16.3% below the benchmark)
```



Cell 96 · TEXT CELL

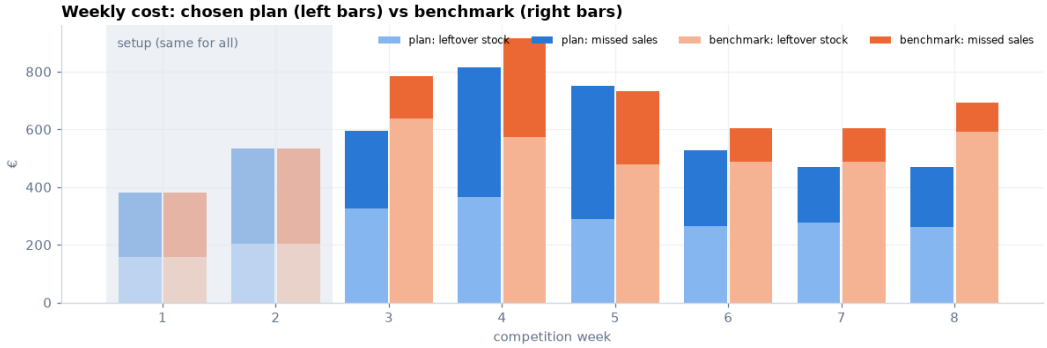
Text: how to read the result honestly (the hindsight caveat), and the weekly cost comparison.

Cell 97 · CODE CELL

What it does. Re-runs the chosen plan week by week (P.run_plan) and charts the weekly costs (leftover and missed sales) against the benchmark.
What you see. Stacked bars per week, and a table.
Why it matters. Shows where the saving comes from: much less leftover stock.

Code (start)

```
plan = P.run_plan(initial, comp_demand, rounds, P.k_vector(disp_all, plan_model), plan_model, plan_service, cos
ts,
                  cfg.policy.samples, SEED)
plots.weekly_costs(plan.weekly, bench_weekly)
pd.DataFrame({"plan cost": plan.weekly["cost"], "benchmark cost": bench_weekly["cost"]}).round(1)
```



18 · Stock outlook: the next order

Cell 98 · TEXT CELL

Heading: step 18, stock outlook. What to order after the data ends.

Cell 99 · CODE CELL

What it does. Builds a table for every store-product: stock on the shelf, stock on its way, forecast, order now, and the chance of running out in weeks 9, 10 and 11.
What you see. Order 1,162 units for 271 products; 19 products have a high risk next week.
Why it matters. Turns the analysis into a concrete action list for the planner.

Code (start)

```
out = P.outlook_table(panel.index, classes["pattern"].to_numpy(), plan, P.forecast_matrix(rounds, plan_model, 9
))
print(f"Order now: {out['order now'].sum():,.0f} units for {(out['order now'] > 0).sum()} store-products")
print(f"High risk (>50%) of an empty shelf next week: {(out['risk wk 9'] > .5).sum()} store-products")
out.sort_values("risk wk 9", ascending=False).head(10).round(2)
```

Output

```
Order now: 1,162 units for 271 store-products
High risk (>50%) of an empty shelf next week: 19 store-products
```

19 · Save the artefacts for the dashboard

Cell 100 · TEXT CELL

Text: weeks 9 and 10 are already decided; the new order protects week 11. Heading: step 19, save.

Cell 101 · CODE CELL

What it does. Saves every result (tables, forecasts, scores, time-series results) to the artifacts folder and lists the files.
What you see. "Saved N files" to the artifacts folder.
Why it matters. The dashboard reads these files, so the dashboard and the notebook always agree.

Code (start)

```
# Tables
files = {"audit.csv": audit, "leaderboard.csv": lb, "leaderboard_h.csv": lb_h, "leaderboard_pattern.csv": lb_p,
         "leaderboard_competition.csv": lb_comp, "calibration.csv": calib, "competition_scores.csv": comp_score
s,
         "benchmark_weekly.csv": bench_weekly, "outlook.csv": out, "error_whiteness.csv": whiteness,
         "tuning.csv": pd.concat([tuning, comp_grid]), "importance.csv": imp.rename("gain").rename_axis("featur
e"),
         "season.csv": season.rename("factor").rename_axis("iso_week")}
for name, table in files.items():
… (38 more lines)
```

Output

```
Saved 24 files to C:\Users\sanya\OptiChain\artifacts
```

20 · Conclusions

Cell 102 · TEXT CELL

Heading: step 20, conclusions.

Cell 103 · CODE CELL

What it does. Prints the conclusions with the real numbers filled in automatically.
What you see. Four lines: time-series result, forecasting, stock policy, final result.
Why it matters. Numbers come from the run itself, so the text can never be out of date.

Code (start)

```
print(f"Time series: the network total is stationary and seasonal (strength {s_net['seasonal strength']:.2f});
"
      f"best model {best_net['model']}; white noise after GARCH. Per store-product ARIMA: {wn:.0%} white noise.
")
print(f"Forecasting: {M.MODEL_LABELS[best_fc]} is the most accurate in the backtest "
      f"({lb.iloc[0]['Score %']:.1f}% vs {bench_score:.1f}% for the benchmark forecast).")
print(f"Stock policy: Monte Carlo newsvendor, service level {plan_service} tuned on past weeks.")
print(f"Result: {M.MODEL_LABELS[plan_model]} plan €{plan_score:,.1f} vs official benchmark €{bench_official:,.1
f} "
      f"({1 - plan_score / bench_official:.1%} lower); winner €{cfg.competition.winner_score:,.1f}.")
```

Output

```
Time series: the network total is stationary and seasonal (strength 0.93); best model SARIMA(2, 0, 0)x(1, 0, 0)[52
]; w…
Forecasting: LightGBM (global ML) is the most accurate in the backtest (57.1% vs 63.7% for the benchmark forecast)
.
Stock policy: Monte Carlo newsvendor, service level 0.75 tuned on past weeks.
Result: Ensemble plan €3,626.4 vs official benchmark €4,334.0 (16.3% lower); winner €3,763.2.
```

Cell 104 · TEXT CELL

Text: limitations and next steps, and the data citation.

The whole notebook in one table

Each step, the cells that do it, and its key result.

Step	Cells	Key result
0 · Setup	1-2	tools loaded, seed fixed
1 · Data collection	3-11	599 store-products × 165 weeks; files consistent; 43% zero weeks
2 · Exploratory data analysis	12-30	strong Christmas peak; about 1 unit a week typical; not normal; stores 0-59% out of stock
3 · Dealing with missing values	31-40	pre-launch and out-of-stock weeks set aside (100% of out-of-stock weeks = 0 sales)
4 · Stationarity: the ADF test	41-49	ADF: total stationary; 94% of products stationary
8 · Identify the model: AR, MA, ARMA, ARIMA or SARIMA?	50-68	the total looks like SARIMA
12 · Forecasting models	69-71	10 methods
13 · Backtest: which model forecasts best?	72-81	LightGBM most accurate (57.1% vs 63.7% benchmark)
14 · Uncertainty: how wrong can a forecast be?	82-97	negative binomial spread; ranges slightly generous
18 · Stock outlook: the next order	98-101	order 1,162 units for 271 products
20 · Conclusions	102-104	conclusions